

GAIA

Monitoramento de plantas com câmera, sensores e IA

Protótipo embarcado: nó de captura, servidor de inferência e documentação

Miguel Pereira Thiago Almeida Leticia Sabadini Gabriel Borges

Multivix Serra

<https://github.com/MrWlker/gaia-project>

Setembro de 2026

Resumo

O GAIA é um protótipo embarcado de fenotipagem de plantas. Um nó de captura, formado por um Raspberry Pi 3, uma câmera Intel RealSense D435 e sensores de ambiente, fotografa o canteiro em intervalos e grava cada captura em disco antes de enviá-la. Um servidor de inferência, num computador com GPU, encontra as plantas na imagem, reconhece cada uma pela posição ao longo do tempo, estima o ambiente no ponto de cada planta e classifica espécie, condição, agente, órgão e hidratação com um modelo multi-cabeça sobre um tronco ConvNeXt-Base, que recebe também as leituras de temperatura e umidade. Este documento descreve a arquitetura, a rede, o servidor, o nó de captura, a preparação do Raspberry Pi e o caminho para construir outros protótipos a partir do código. O código e esta documentação estão no repositório; o modelo pré-treinado é publicado à parte, e o código de treino e os dados acompanham o artigo.

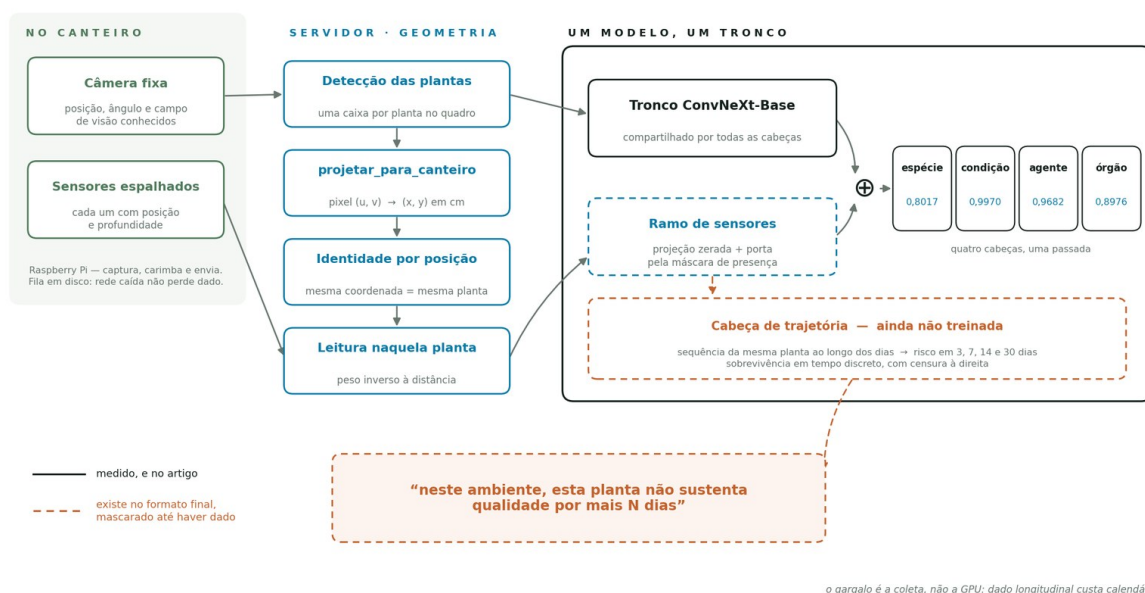


Figura 1: Arquitetura do GAIA: o nó no canteiro, a geometria no servidor e o modelo com tronco compartilhado, ramo de sensores e cabeças por pergunta. Tracejado: parte que existe no formato final e aguarda dado.

Sumário

1	Visão geral	4
1.1	O problema	4
1.2	Onde cada coisa roda	4
1.3	O que acontece numa captura	4
2	Resultados em funcionamento	4
3	A rede	7
3.1	Entradas e saídas	7
3.2	Limites que as respostas não escondem	8
4	Etapas atuais	8
5	Servidor e inferência	8
5.1	Subir	8
5.2	Cadastro de canteiro, câmeras e sensores	9
5.3	O pipeline do /ingest	9
5.4	API	9
5.5	Usar só a inferência	10
5.6	Custo	10
6	Estação de captura e interface	10
6.1	A interface	10
6.2	Configuração	11
6.3	Sensores plugáveis	11
6.4	RealSense D435	11

7	Raspberry Pi	11
7.1	Sistema e instalação offline	11
7.2	Rede	11
7.3	Sem tela nem teclado	12
7.4	Ligação do DHT11	12
7.5	Testar sem hardware	12
8	Criando outros protótipos	12
9	Publicação do modelo	13
	Referências	13

1 Visão geral

1.1 O problema

Classificar uma foto de folha não basta. O objetivo do GAIA é acompanhar **cada planta ao longo do tempo** — a mesma planta hoje, amanhã e na semana seguinte — junto do ambiente em que ela vive, para no futuro prever a trajetória de saúde dela. Isso impõe três decisões de projeto que aparecem em todo o código.

Identidade da planta pela posição. A câmera é fixa e a planta não anda. Cada detecção é projetada do pixel para uma coordenada (x, y) em centímetros no canteiro, e duas detecções a menos de 20 cm uma da outra, em dias diferentes, são a mesma planta. Não há rede de rastreamento nem cadastro manual. A origem das coordenadas é o controlador do canteiro; câmeras e sensores são cadastrados pelo deslocamento até ele.

O sensor que vale é o mais próximo da planta. Com vários sensores espalhados, a leitura atribuída a cada planta é a interpolação por inverso da distância no ponto dela, não a média do canteiro. Cada leitura é uma linha por sensor por instante, porque uma coluna anônima “solo1” não sabe a que pedaço do canteiro pertence.

Dado longitudinal não se refaz. Perder uma semana de captura custa uma semana de calendário. Por isso o nó grava primeiro em disco e envia depois; se o servidor cai, a fila cresce e escoar quando ele volta. Leitura que falha vira *ausente*, nunca um valor inventado.

1.2 Onde cada coisa roda

O Raspberry Pi 3 tem 1 GB de RAM e um Cortex-A53 sem aceleração útil para convolução; só o ConvNeXt pede centenas de megabytes de ativação a 384 px. Por isso **nenhuma rede neural roda no Pi**: ele captura, carimba, guarda e envia. O mesmo código do nó roda num PC, o que permite desenvolver a interface e a inferência com a câmera no notebook antes de o Pi estar pronto.

1.3 O que acontece numa captura

1. O nó pede à câmera o próximo quadro alinhado e grava na fila três arquivos: a cor (JPEG), a profundidade (PNG de 16 bits, em milímetros, alinhada à cor) e um JSON com as leituras e a intrínseca de fábrica da câmera.
2. Envia ao servidor (POST `/ingest`); se falhar, a captura fica na fila.
3. O servidor encontra as regiões de vegetação e, para cada uma: projeta no canteiro, acha ou cria a planta, interpola os sensores ali, classifica o recorte e calcula o Grad-CAM da condição.
4. Grava tudo e desenha a imagem anotada (caixas, máscara e foco).
5. A resposta volta ao nó e aparece na interface.

2 Resultados em funcionamento

A Figura 2 mostra capturas reais da bancada, feitas com a D435 e anotadas pelo servidor. As caixas são as plantas encontradas; a cor indica a condição (verde saudável, âmbar incerto, vermelho doente); o vermelho translúcido marca onde a rede se apoiou para decidir “doente”.

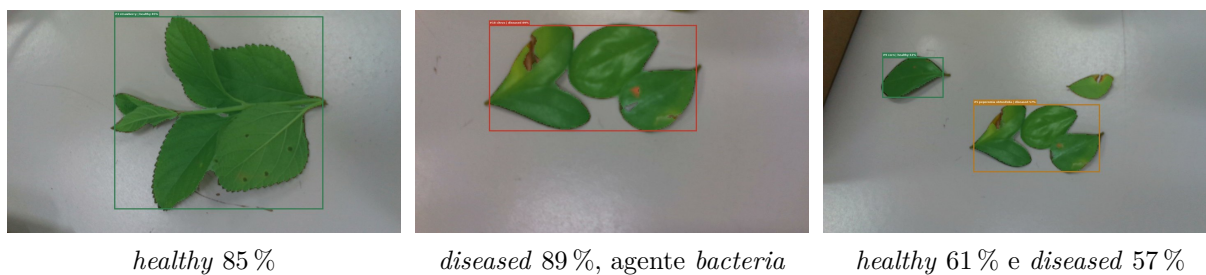


Figura 2: Capturas reais da bancada com a RealSense D435, anotadas pelo servidor.

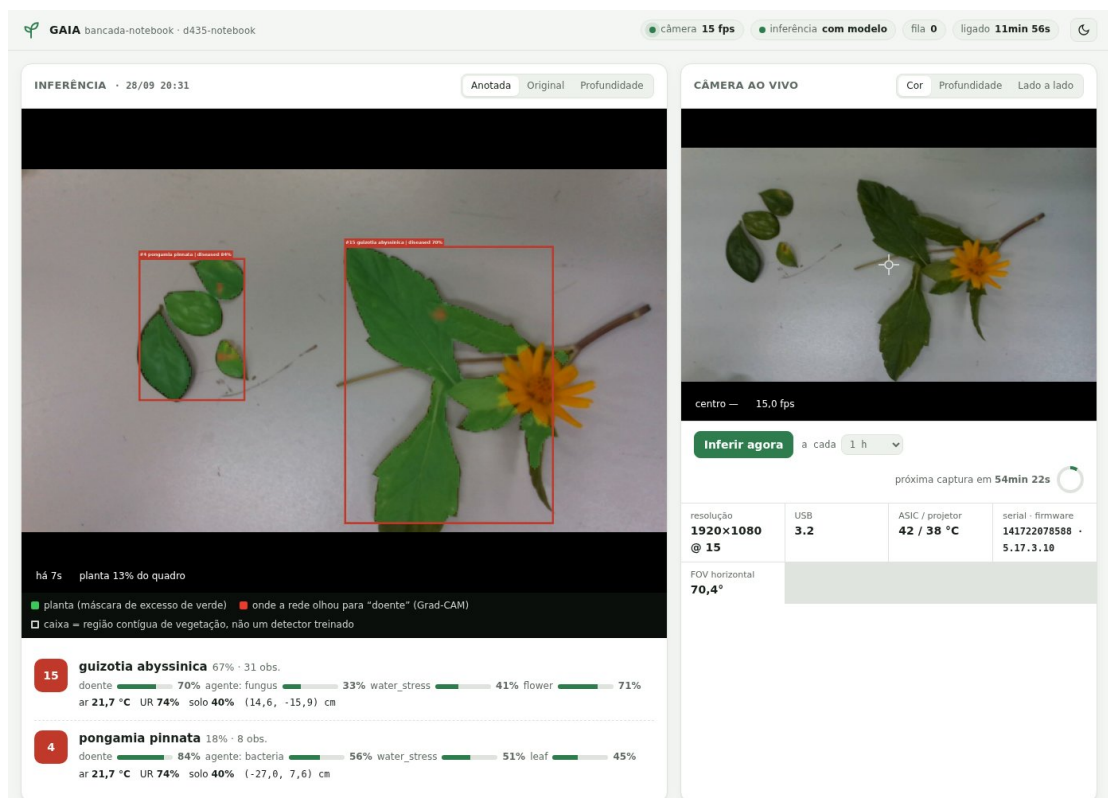
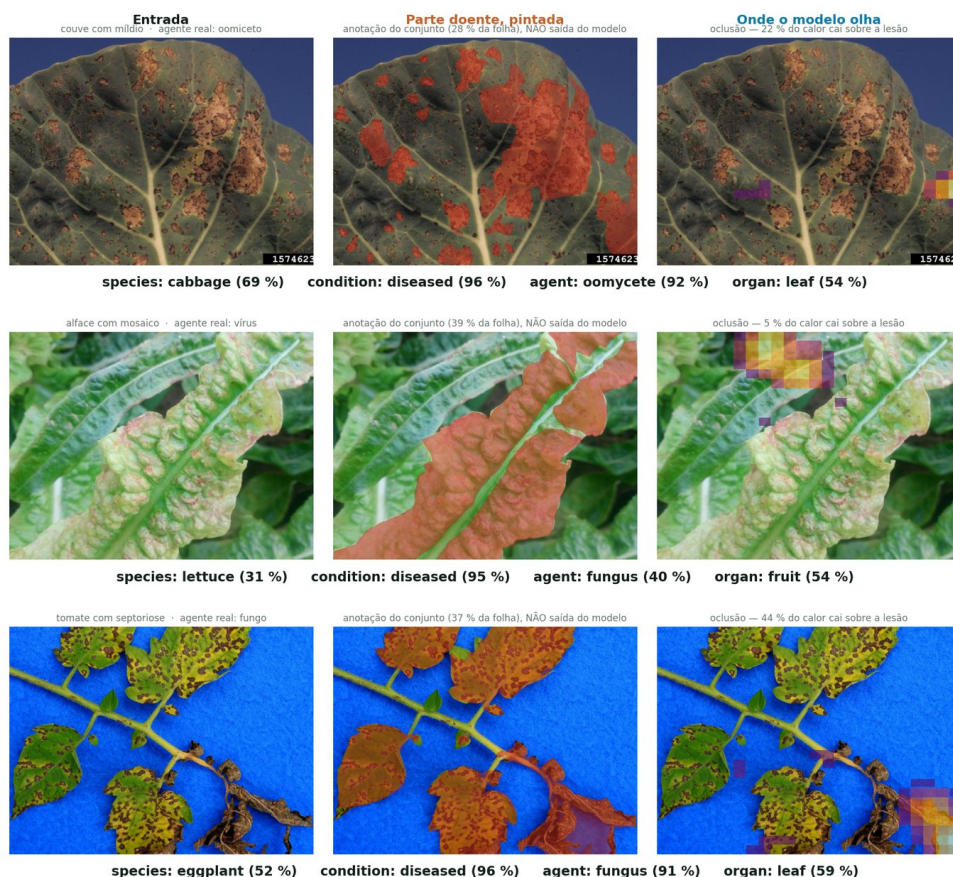


Figura 3: Interface web do nó: a inferência como painel principal e a câmera ao vivo ao lado, com resolução, USB, temperatura do ASIC da câmera e o intervalo de captura.



O eixo AGENTE acerta 2 dos 3 — confiança de 92, 40, 91 % — e a condição acerta os 3.

É o que os números já diziam: no PlantSeg o agente vale 0,9041 e a espécie 0,6843. E ele não custou nenhuma anotação nova — sai do nome da pasta de origem.

Figura 4: Resultado qualitativo do modelo: a entrada, a parte doente pintada, o mapa de oclusão e as cabeças com a confiança de cada uma.



Cena sintetizada por mosaico porque ela não existe nos conjuntos públicos: o PlantDoc-OD tem 1 imagem em 2.566 com mais de uma espécie no quadro.

Figura 5: Cena com várias plantas: máscara por instância, projeção de cada planta para o canteiro e a umidade estimada no ponto de cada uma.

3 A rede

Um modelo só, com um tronco compartilhado e uma cabeça por pergunta (Figura 6). Os sensores entram por um ramo à parte, somado às features da imagem **apenas nas cabeças em que o ambiente importa**.

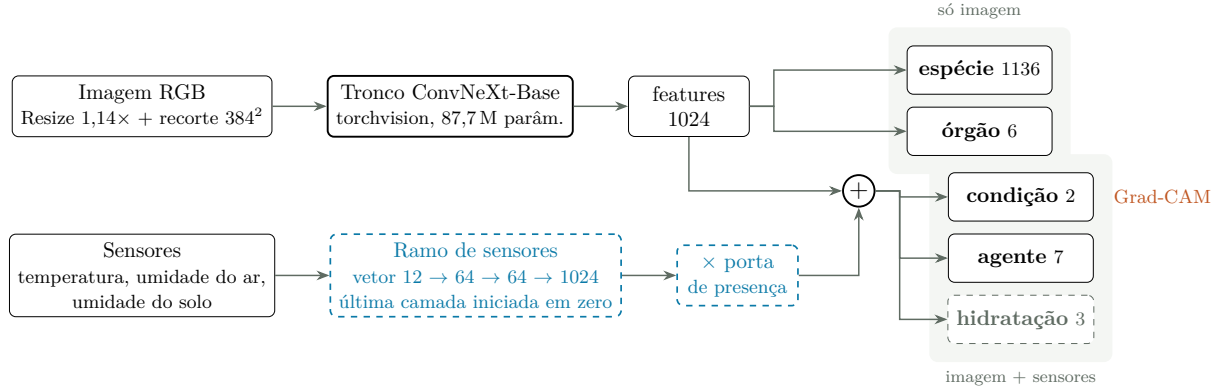


Figura 6: A rede GAIA. Espécie e órgão usam as features puras da imagem; condição, agente e hidratação usam imagem somada ao ramo de sensores. Tracejado: parte que existe no modelo mas aguarda dado (o ramo de sensores e a cabeça de hidratação).

- **Espécie e órgão** dependem só da aparência e usam as features puras.
- **Condição, agente e hidratação** usam imagem e sensores.
- O ramo de sensores nasce **neutro**: a projeção final começa em zero e a porta zera a contribuição quando não há leitura. O modelo é numericamente idêntico ao só-imagem até existir dado de sensor que ensine alguma coisa.
- Sensor ausente é ausente (flag 0), nunca zero.

3.1 Entradas e saídas

Imagem. RGB, redimensionada para 1,14× o tamanho de entrada e recortada no centro (384 px no ConvNeXt publicado). A borda da imagem não é vista pela rede, e quem pinta explicação sobre a foto precisa levar isso em conta.

Sensores. Temperatura (°C), umidade do ar (%) e umidade do solo (%), todos opcionais. Viram um vetor de 12 posições: valor normalizado, flag de presença, desvio da faixa ideal da espécie e flag do desvio.

eixo	classes
species	1136 espécies
condition	<i>diseased, healthy</i>
agent	<i>none, fungus, oomycete, bacteria, virus, pest, abiotic</i>
organ	<i>leaf, flower, fruit, bark, habit, branch</i>
hydration	<i>hydrated, water_stress, waterlogged</i> (ainda sem rótulo)

Tabela 1: As cabeças de saída; cada uma tem softmax próprio.

O checkpoint lista em `trained_axes` as cabeças que tiveram rótulo no treino. As demais existem, mas a saída delas é ruído, e o preditor as marca como não treinadas.

3.2 Limites que as respostas não escondem

- **Espécie:** a confiança é baixa fora das fotos de laboratório (folha isolada, fundo limpo). No canteiro, a espécie é sugestão; a identidade da planta vem da posição.
- **Caixas:** saem da máscara de vegetação, não de um detector treinado.
- **Vermelho do Grad-CAM:** mostra onde a rede se apoiou para “doente”, não mede lesão; o modelo nunca viu máscara de lesão no treino.
- **Ramo de sensores:** nenhuma imagem de treino tinha leitura de sensor. Até existir a série da horta, o efeito das leituras sobre a predição é pequeno.

4 Etapa atual

Modelo

- Classificação multi-cabeça treinada (espécie, condição, agente, órgão)
- Tronco ConvNeXt-Base a 384 px, robusto a borrão, baixa resolução e ruído
- Ramo de sensores na arquitetura, neutro até haver dado
- Explicação por Grad-CAM na cabeça de condição
- Cabeça de hidratação com rótulo
- Ramo de sensores treinado com leituras da horta
- Cabeça de trajetória: “não sustenta qualidade por mais N dias”
- Detector de plantas treinado
- Modelo público (privado até o artigo)
- concluído □ pendente

Sistema

- Servidor com identidade por posição e sensores por distância
- Nó com D435 (cor + profundidade), fila em disco e interface web
- Sensores plugáveis
- Inferência contínua na bancada, a cada 10 s
- Imagem do SD com instalação offline, validada em ARM64 emulado
- Validação no Raspberry Pi 3 físico
- Coleta longitudinal na horta
- Artigo

5 Servidor e inferência

5.1 Subir

```
pip install -r requirements.txt          # torch com CUDA: a versao da sua GPU
python3 servidor/baixar_modelo.py        # uma vez; modelo -> modelo/
cd servidor                               # 0.0.0.0:8077, modelo de ../modelo
./iniciar.sh                             # + uma estacao simulada em :8080
./iniciar.sh --com-simulada
```

variável	padrão	para quê
GAIA_MODELO	../modelo	pasta ou .pt; vazio = só ingestão, sem classificar
GAIA_BANCO	sqlite:///gaia.db	qualquer URL do SQLAlchemy
GAIA_MIDIA	midia	imagens, profundidade e anotações
GAIA_REFERENCIA	pasta do modelo	tabela de faixas ambientais por espécie

Sem modelo, o servidor funciona em **ingestão pura**: localiza plantas e guarda a série e os sensores, o que permite começar a coletar antes de o modelo final existir.

5.2 Cadastro de canteiro, câmeras e sensores

```
python3 cadastrar.py local canteiro-1
python3 cadastrar.py camera d435-frontal --local 1 --x 0 --y -60 --z 80 --pitch -45
--fov 69.4
python3 cadastrar.py sensor ar --local 1 --canal temp --tipo temperature --central
python3 cadastrar.py sensor solo-a --local 1 --canal soil1 --tipo soil_moisture --x
20 --y 30
python3 cadastrar.py listar
```

- Distâncias em cm a partir do controlador; ângulos em graus; **pitch** negativo olha para baixo. A estação mostra o FOV exato, lido da câmera.
- **-canal** é o nome que o nó envia. Canal não cadastrado é ignorado, então a temperatura da CPU pode ir junto sem poluir o dado da planta.
- **-tipo** diz o que a leitura significa para o modelo. Só **temperature**, **air_humidity** e **soil_moisture** entram no vetor de sensores.
- Bancada e canteiro real ficam em **locais diferentes**: dado de teste misturado à série de uma planta real não se separa depois.

5.3 O pipeline do /ingest

1. **Plantas na imagem.** Máscara de vegetação pelo índice de excesso de verde (ExG) com limiar de Otsu e um piso absoluto de 0,10. Sem o piso, uma cena sem planta ainda teria “metade verde”; medido com a D435, o teto branco chega a $p_{99} = 0,085$ e a folha passa de 0,5. Folhas próximas são agrupadas numa planta; cada grupo com mais de 1,5% do quadro vira uma caixa (no máximo seis). Sem vegetação, não há caixa nem classificação.
2. **Posição.** A base da caixa é projetada no plano do solo. Se o raio não cruza o solo, a detecção é guardada sem posição, em vez de inventar uma.
3. **Identidade.** A planta mais próxima a menos de 20 cm, ou uma nova; a posição é refinada pela média a cada observação.
4. **Sensores no ponto**, por inverso da distância; o sensor central é o fallback.
5. **Classificação** do recorte com o vetor de sensores daquela planta.
6. **Grad-CAM** da cabeça de condição no recorte, limitado à máscara de planta e com opacidade proporcional a $P(\text{doente})$.
7. **Imagem anotada**, servida em /midia/....

5.4 API

O campo meta do /ingest:

```
{
  "no": "canteiro-1",
  "camera": "d435-frontal",
  "instante": "2026-09-28T22:44:06+00:00",
  "imagem": "d435-frontal_20260928T224406Z.jpg",
  "leituras": {"temp": 24.1, "umid_ar": 66.0, "soil1": null},
```

método	rota	o quê
POST	/ingest	multipart: imagem, meta (JSON), profundidade (opcional)
GET	/saude	estado do servidor e dos modelos
GET	/plantas	plantas conhecidas, posição e número de observações
GET	/planta/{id}/serie	série temporal de uma planta
GET	/midia/{caminho}	imagens gravadas e anotadas
GET	/docs	documentação interativa

```
"camera_info": {"intrinseca_cor": {"fx": 615.3, "fy": 615.1, "cx": 322.5, "cy": 240.7}}
```

A câmera precisa estar cadastrada; sem extrínseca não há como projetar, e a resposta é 404. Leitura null significa “sem leitura”. A resposta traz, por planta, a posição, a espécie, a condição, o agente, a hidratação, o órgão (cada um com a confiança), o ambiente estimado no ponto e a caixa, além do caminho da imagem anotada.

5.5 Usar só a inferência

```
from gaia.preditor import Predictor
p = Predictor("modelo/")
r = p.predict("foto.jpg", readings={"temperature": 27.4, "air_humidity": 68})
print(r["condition"]["label"], r["condition"]["confidence"])
```

5.6 Custo

Numa RTX 3050 de 4 GB, o ConvNeXt a 384 px carrega em cerca de 4 s e classifica em cerca de 0,35 s por recorte; o Grad-CAM custa uma passada para trás por recorte. Com até seis plantas por quadro, um ciclo de 10 s cabe com folga. Nesse ritmo, as imagens somam cerca de 700 MB por dia.

6 Estação de captura e interface

```
cd estacao
python3 estacao.py --config estacao-notebook.json    # PC com a D435 no USB
python3 estacao.py --config estacao.json            # Pi (via systemd)
python3 estacao.py --simular                        # sem camera nem sensores
```

A estação usa só a biblioteca padrão do Python para o HTTP: num Pi de 1 GB cada dependência pesa, e o FastAPI puxaria o `pydantic-core`, que em armv7 compila em Rust por horas. As dependências de fato são `numpy`, `Pillow`, `requests` e `pyrealsense2`.

6.1 A interface

- **Inferência**, o painel principal: a última imagem, trocando sozinha a cada ciclo, com as abas anotada, original e profundidade, e abaixo cada planta com espécie, condição, agente, hidratação, órgão e o ambiente no ponto.
- **Câmera ao vivo**: MJPEG de cor, profundidade ou lado a lado; distância no centro; temperatura do ASIC; botão “Inferir agora” e intervalo de 5 s a 1 h.
- **Sensores, histórico** (1 h a 7 dias), **capturas** e **sistema**.

A página é um único arquivo, sem CDN, porque o Pi pode estar sem internet.

6.2 Configuração

O campo `servidor` é uma lista: vale o primeiro que responder `/saude`, de modo que o mesmo arquivo serve no cabo, no hotspot e na VM. O modo `-simular` troca a câmera por uma cena sintética e cada sensor de hardware por um simulado nos mesmos canais, gravando como outra câmera, numa pasta à parte.

6.3 Sensores plugáveis

tipo	observação
cpu	temperatura do SoC
realsense	ASIC e projetor da D435; acusa câmera cozinhando ao sol
dht11, dht22	temperatura e umidade do ar num GPIO digital
ds18b20	1-Wire, lido pelo sysfs
mcp3008	umidade do solo por ADC SPI (o Pi não tem entrada analógica)
comando	qualquer programa que imprima um número
simulado	ciclo diário com ruído e falhas ocasionais

6.4 RealSense D435

Um pipeline só: uma thread guarda o último quadro para o vídeo ao vivo, e a captura pede o próximo quadro **alinhado**, com a profundidade reprojeta na geometria da cor. No PC, a cor vai a 1920×1080 e a profundidade a 1280×720; em USB 2, caso do Pi 3, a estação força 640×480 a 15 fps. A captura guarda a intrínseca de fábrica, o que abre caminho para a projeção sem trena, e a profundidade por pixel, a primeira medida métrica da planta.

7 Raspberry Pi

7.1 Sistema e instalação offline

O `pyrealsense2` só publica wheel aarch64 para Python 3.8 e 3.9, então o sistema é o Raspberry Pi OS Lite 64-bit *bullseye* (Python 3.9). O *bullseye* saiu de suporte em 2026-08, e o repositório de segurança já anuncia pacotes removidos. Por isso a instalação é **100 % offline**: no PC, `montar_offline.sh` baixa numpy, Pillow, pyrealsense2, requests e a pilha do DHT como wheels aarch64, além da `libusb` de que o `pyrealsense2` precisa; no Pi, `instalar_pi.sh` os extrai sem `apt`, `venv` nem `pip`.

```
cd estacao/imagem
./montar_offline.sh
GAIA_ETH_IP=192.168.123.144/24 GAIA_ETH_GW=192.168.123.99 ./gerar_imagem.sh
lsblk -o NAME,SIZE,MODEL,TRAN # confira o dispositivo
sudo dd if=gaia-pi.img of=/dev/sdX bs=4M conv=fsync status=progress
```

No primeiro boot, a imagem expande a partição, configura hostname, usuário, SSH e rede, copia o código e instala a estação sozinha. O IP pode ser trocado sem regravar o cartão, editando `gaia-rede.conf` na partição de boot.

7.2 Rede

O caminho mais simples é o **cabo direto no PC**: o PC vira 192.168.123.99 e compartilha a internet do Wi-Fi dele (`servidor/cabo.sh`). O DNS padrão do Pi é o próprio gateway, porque redes de laboratório costumam bloquear servidores públicos como 1.1.1.1 e 8.8.8.8. Também há um hotspot em 2,4 GHz (`servidor/hotspot.sh`).

7.3 Sem tela nem teclado

- `servidor/achar_pi.sh -f` varre a rede e diz a etapa: fora da rede, instalando (só SSH) ou interface no ar. Ignora celulares pelo endereço MAC.
- Pelo SSH: `journalctl -fu gaia-instalar` e `systemctl status gaia-estacao`.
- Sem acesso nenhum: tirar o cartão e ler, no PC, os logs `gaia-firstrun.log`, `gaia-instalacao.log` e `gaia-rede.log` da partição de boot.

7.4 Ligação do DHT11

DHT11	Raspberry Pi 3 B/B+	observação
VCC (+)	pino 1 — 3,3 V	não usar 5 V: o GPIO só aguenta 3,3 V
DATA	pino 7 — GPIO4	10 k Ω entre DATA e VCC no sensor de 4 pinos
GND (—)	pino 6 — GND	

O módulo de três pinos numa plaquinha já tem o resistor de pull-up. A D435 no Pi 3 puxa até cerca de 700 mA: use fonte de 2,5 A e, de preferência, um hub USB alimentado.

7.5 Testar sem hardware

Uma VM QEMU (máquina virt, Cortex-A53, 4 núcleos, 1 GB) reproduz o orçamento do Pi 3 e aceita a D435 pelo xHCI. Para validar só a instalação offline, basta um container `arm64v8/python:3.9-slim-bullseye` com `-network none`; foi assim que se descobriu a dependência da `libusb`.

8 Criando outros protótipos

O contrato entre as partes é pequeno: o nó envia `POST /ingest` com uma imagem e um JSON de leituras por canal, e o servidor sabe onde cada câmera e cada sensor estão.

Um sensor novo. Sem código, pelo tipo comando; com código, uma classe em `estacao/sensores.py`:

```
class BH1750(Driver):
    def descrever(self):
        return [(self.cfg.get("canal", "luz"), "Luminosidade", "lx")]

    def ler(self):
        try:
            return {self.descrever()[0][0]: ler_bh1750(self.cfg.get("endereco", 0x23))}
        except Exception:
            return {self.descrever()[0][0]: None}    # falha = None

TIPOS["bh1750"] = BH1750
```

Para o modelo usar a leitura, ela precisa ser de um tipo que ele conhece; um tipo novo exige retrainar o ramo de sensores.

Um nó em outro hardware. Qualquer dispositivo capaz de um POST multipart é um nó (ESP32-CAM, celular, outro computador de placa única). Regras que valem para todos:

1. cadastrar a câmera no servidor, com posição, ângulos, FOV e resolução;
2. gravar antes de enviar;

3. manter o relógio certo, porque o instante vem do nó e um Pi sem RTC acorda com a data da última vez que desligou;
4. enviar `null` para “sem leitura”: zero é leitura.

Mais de uma câmera. Cada uma com a própria posição. Como a identidade é pela coordenada no canteiro, a mesma planta vista por duas câmeras cai no mesmo identificador.

Outra forma de achar plantas. Um detector treinado entra implementando a mesma assinatura de `caixas_pela_mascara`, que devolve a máscara e as caixas.

Outro modelo. O preditor lê do checkpoint a arquitetura, o tamanho de entrada, as classes e o ramo de sensores. Qualquer checkpoint no mesmo formato funciona sem mudar código; uma MobileNet roda em CPU e serve para servidor sem GPU.

9 Publicação do modelo

O modelo não está no repositório de código. Ele é exportado só com o que a inferência usa e publicado no Hugging Face:

```
python3 ferramentas/exportar_modelo.py best.pt publicar/ species_env.csv
huggingface-cli upload <usuario>/<repositorio> publicar/ . --repo-type model
python3 servidor/baixar_modelo.py --repo <usuario>/<repositorio>
```

São publicados os pesos com a configuração e as classes, a lista de classes em JSON, a tabela de faixas ambientais por espécie e o cartão do modelo. O arquivo `.pt` é um pickle do PyTorch: carregue apenas checkpoints de fonte confiável.

Referências

Modelo e métodos

- Z. Liu, H. Mao, C.-Y. Wu, C. Feichtenhofer, T. Darrell, S. Xie. *A ConvNet for the 2020s*. CVPR, 2022.
- R. R. Selvaraju, M. Cogswell, A. Das, R. Vedantam, D. Parikh, D. Batra. *Grad-CAM: Visual Explanations from Deep Networks via Gradient-based Localization*. ICCV, 2017.
- D. M. Woebbecke, G. E. Meyer, K. Von Bargen, D. A. Mortensen. *Color indices for weed identification under various soil, residue, and lighting conditions*. Transactions of the ASAE, 38(1), 1995.
- N. Otsu. *A threshold selection method from gray-level histograms*. IEEE Transactions on Systems, Man, and Cybernetics, 9(1), 1979.

Conjuntos de dados usados no treino (não distribuídos)

- D. P. Hughes, M. Salathé. *An open access repository of images on plant health to enable the development of mobile disease diagnostics*. arXiv:1511.08060, 2015 (PlantVillage).
- D. Singh, N. Jain, P. Jain, P. Kayal, S. Kumawat, N. Batra. *PlantDoc: A Dataset for Visual Plant Disease Detection*. CoDS-COMAD, 2020.
- C. Garcin et al. *Pl@ntNet-300K: a plant image dataset with high label ambiguity and a long-tailed distribution*. NeurIPS Datasets and Benchmarks, 2021.
- T. Wei et al. *PlantSeg: A Large-Scale In-the-wild Dataset for Plant Disease Segmentation*. arXiv:2409.04038, 2024.
- GBIF.org — Global Biodiversity Information Facility, ocorrências com imagem.

Hardware e software

Intel RealSense D400 Series Product Family Datasheet; librealsense e pyrealsense2. PyTorch e torchvision; FastAPI; SQLAlchemy; Raspberry Pi OS.